Natural Language Processing compact cheatsheet

STEP 1 NLU · Language and meaning

1.1 What language is

Working definition Language is a systematic, symbolic and structured system that starts as thought, organises it, and lets it be rebuilt in another mind.

Six levels Phonetics, phonology, morphology, syntax, semantics, pragmatics. Each combines the units of the level below.

Two functions Symbolic: words stand for concepts, not directly for things (semiotic triangle). Interactive: a speaker encodes and a hearer reconstructs, never an exact copy.

Form and meaning A symbol is a conventional form meaning pair. Unicorn has a sense without a reference. TAM marks present, past and imagined reality.

Effort and clarity Zipf: least effort, f(r) ≈ C ÷ r. Grice: quantity, quality, relation, manner.

Recursion A clause inside a clause without limit. Everett claimed Pirahã lacks it, a challenge to Hauser, Chomsky and Fitch. Still debated.

Working definition from the lecture

Language is a **systematic, symbolic and structured** system that (1) starts as thought in the mind, (2) organises chaotic ideas into structure, and (3) lets those ideas be sent out, carried and rebuilt in another mind.

The key claim

Language does not directly represent the external world. It represents our **conceptualised experience** of the world.

The communication chain

Thought → linguistic representation → signal → linguistic representation → reconstructed thought

TRAP Listeners usually also infer "if it does not rain, I will go", which the logic does not say. This gap between logical form and everyday meaning is one reason formal logic alone could not solve language understanding.

EXAM In an essay, link Zipf to the speaker side (minimise effort) and Grice to the shared contract between both sides.

1.2 Ambiguity: why language is hard for machines

Two kinds Lexical: one word, several meanings (bank, over). Structural: one string, several trees (telescope).

Word types Homonymy (unrelated meanings), polysemy (related senses), homophones (same sound), homographs (same spelling).

Lesk Pick the sense whose gloss shares most words with the context. A counting version of Firth.

Catalan growth A verb, object and k prepositional phrases have C(k + 1) trees: 2, 5, 14, 42, 132.

Resolution Rules (right association), treebank statistics (lexical association), contextual neural models. Ask when the cost of a wrong guess is high.

Lecture examples I sat near a bank. The cat jumped over the wall. John saw Mary with a telescope. Understands you like your mother. Copy the local patient files.

Simplified Lesk

$$\text{sense}^* = \operatorname{argmax}_s |\text{context}(w) \cap \text{gloss}(s)|$$

Catalan numbers

$$C_n = \frac{1}{n+1} \times (2n \text{ choose } n) = \frac{(2n)!}{(n+1)!\, n!}$$

EXAM If an exam asks you to classify an example, ask one question: does any single word have two meanings? If yes, lexical. If every word is clear but the grouping can change, structural.

1.3 Can machines understand language?

Marr Computational (what and why), algorithmic (how, representation), implementational (where). Design top down, validate bottom up.

Turing test Operational, text only, not reproducible. Qualia are outside it. The Total Turing Test adds perception and action, seven capabilities.

ELIZA and SHRDLU ELIZA: keyword patterns, no understanding. SHRDLU: world model, reasoning, action and clarifying questions in a blocks world.

Fields and pipeline NLP builds tools, CL models cognition, NLU is the shared goal. Corpus and annotation, model, then validation.

Metrics P = TP ÷ (TP + FP), R = TP ÷ (TP + FN), F1 = 2PR ÷ (P + R). BLEU = BP · exp(Σ w log p). Surprisal $-\log_2 P$. Dependency length.

Form against meaning Firth and Shannon support data. The octopus test, Ettinger, symbol grounding and the Chinese room warn that form is not meaning.

Precision, recall, F1

$$\text{P} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{R} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1} = \frac{2\text{PR}}{\text{P} + \text{R}}$$

BLEU

$$\text{BLEU} = \text{BP} \cdot \exp(\Sigma_n w_n \log p_n)$$

TRAP The professor calls jumping straight to hardware and algorithms "the rush that all computer engineers make". A purely data driven language model is tuned at levels 2 and 3 without a clear level 1 answer to what understanding is.

STEP 2 NLU · Text as data

2.1 Sentences, tokens and vocabularies

Pipeline Sentence segmentation, tokenisation, stemming or lemmatisation, POS tagging, counting.

Tokens and types Token = an occurrence, type = a distinct word. N() and B() in FreqDist. TTR = V ÷ N falls as text grows.

Punkt Unsupervised sentence splitter that learned abbreviations. Kept "Dr. jack" together: 3 sentences in the notebook.

Treebank tokens Punctuation split off, Don't → Do n't, It's → It 's. 32 tokens for the notebook paragraph against 26 from split().

Subwords BPE merges the most frequent adjacent pair again and again, so no word is out of vocabulary.

Tokens and types

N = number of tokens, V = |vocabulary| = number of types

EXAM Every SRILM file in chapter 6 has one sentence per line with tokens separated by spaces. That format is the result of exactly the segmentation and tokenisation in this chapter.

2.2 Stems and lemmas

Stem against lemma Stem: rule based chop, may not be a word (leaves → leav). Lemma: dictionary form, uses POS (leaves → leaf, children → child).

Porter measure Word = [C](VC)^m[V]. tree 0, trouble 1, troubles 2. y after a consonant is a vowel.

Notebook results Porter: sportingli, fairli, boy'. Snowball: sport, fair, boy. Exercise list differs only on alumnus.

Lemmatiser needs POS WordNetLemmatizer defaults to noun. loving stays loving, but with pos='v' it becomes love.

Errors and effect Over stemming: university, universe → univers. Under stemming: alumnus and alumni stay apart. Stemming raises recall, may lower precision.

Porter's word shape $[\text{C}]\ (\text{VC})^m\ [\text{V}]$

Precision and recall

$$P = \frac{\text{relevant returned}}{\text{all returned}} \qquad R = \frac{\text{relevant returned}}{\text{all relevant}}$$

EXAM In an exam, write the C and V pattern under the letters first, then count VC pairs. Most mistakes come from forgetting that y after a consonant is a vowel.

TRAP Run a POS tagger first and pass each tag to the lemmatiser. Without the tag, most verbs and adjectives come back unchanged, exactly as in the notebook output above.

2.3 Counting words and tags

FreqDist N() = tokens (outcomes), B() = types (samples), freq(x) = c(x)/N, unseen count = 0. a b a a c b a gives 3 samples, 7 outcomes, freq(a) = 0.5714.

Brown numbers 1,161,192 tokens, 56,057 types, the = 62,713 (5.40%). 472 distinct tags, top tag NN 152,470.

PTB numbers 100,676 tokens, 12,408 types, comma is most common (4,885), the 4,045 (4.02%). 6,592 -NONE- tokens.

Tagsets 8 traditional, 12 universal, 17 UPOS, 45 PTB, about 87 Brown. Fine maps to coarse: ADV 3,171 = RB + RBR + RBS + WRB.

Bigram counts P(w | prev) = c(prev w) / c(prev). From I am Sam: P(am | I) = 2/3.

Relative frequency

$$\text{freq}(x) = \frac{\text{c}(x)}{N}$$

Merging tags adds their counts

$$\text{c(ADV)} = \text{c(RB)} + \text{c(RBR)} + \text{c(RBS)} + \text{c(WRB)}$$

Bigram probability by counting

$$\text{P}(w_i \mid w_{i-1}) = \frac{\text{c}(w_{i-1}\, w_i)}{\text{c}(w_{i-1})}$$

EXAM Exam questions often ask you to "verbally describe how the results differ". Mention size, genre, tagset and the -NONE- tokens, and back each point with one number.

STEP 3 NLU · Information and surprise

3.1 Zipf's law and the shape of vocabulary

Zipf's law f = k/r, f × r ≈ constant, slope −1 on log log axes. f_a/f_b = b/a, so the 50th word is 3 times the 150th.

Brown check the 69,971, of 36,412 (ratio 1.92). f × r stays between about 70,000 and 107,000. Real slope about −1.08.

Coverage Top n share = H_n/H_V, H_n ≈ ln n + 0.5772. V = 50,000: top 10 = 25.7%, top 100 = 45.5%.

Least effort Frequent words are short (law of abbreviation). Language balances speaker effort and listener robustness.

Monkey typing Random keys give a power law with α = 1 − ln(1 − s)/ln M (1.0116 for 27 keys) but no predictability.

Zipf's law $f = \frac{k}{r}$ or $f \times r \approx k$

Ratio rule $\frac{f_a}{f_b} = \frac{b}{a}$

Share of the top n words

$$\text{share}(n) = \frac{H_n}{H_V}, \quad H_n = 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{n}$$

Random typing exponent $\alpha = 1 - \frac{\ln(1-s)}{\ln M}$

EXAM The class question "should the 50th word occur three times as often as the 150th?" has answer yes: $f_{50} \div f_{150} = 150 \div 50 = 3$. On Brown the real ratio is 2,139 ÷ 610 = 3.51, a little steeper, as the slope of −1.08 suggests.

3.2 Surprise, entropy and redundancy

Surprisal I(x) = $-\log_2$ p(x). p = 1 gives 0 bits, each halving adds 1 bit. Bits, nats, hartleys.

Entropy H = $-\Sigma$ p $\log_2$ p = expected surprise. 0 ≤ H ≤ $\log_2$ n. Coin 1 bit, 8 sided die 3 bits, certain event 0.

Codes Fixed length needs ⌈$\log_2$ n⌉ bits. Huffman merges the two least likely. H ≤ $\bar{L}$ < H + 1. Example 1.9 against H 1.846.

Redundancy R = 1 − H/H_max. English about 2.3 of 4.7 bits per letter, so about 50% redundant. u after q carries 0.02 bits.

Limits of H Single symbol entropy ignores order (ABAB has 1 bit). Language needs conditional entropy: context reduces uncertainty.

Surprisal of one outcome

$$I(x) = -\log_2 p(x) = \log_2 \frac{1}{p(x)}$$

Shannon's entropy

$$H(X) = -\sum_x p(x) \log_2 p(x) = \mathrm{E}[\, -\log_2 p(X) \,]$$

Bounds $0 \le H(X) \le \log_2 n$

Source coding theorem $H(X) \le \bar{L} < H(X) + 1$

Redundancy $R = 1 - \frac{H}{H_{\max}}$

3.3 Mutual information, KL and the noisy channel

Joint and conditional H(Y|X) = Σ p(x) H(Y|X=x) = $-\Sigma$ p(x,y) log p(y|x). Chain rule H(X,Y) = H(X) + H(Y|X). Umbrella: 0.971 + 0.621 = 1.592.

Mutual information I = H(Y) − H(Y|X) = H(X) + H(Y) − H(X,Y) = Σ p(x,y) log p(x,y)/(p(x)p(y)). Symmetric, ≥ 0, 0 iff independent. Umbrella 0.378.

PMI $\log_2$ c(xy)N/(c(x)c(y)). New Delhi 8.64, new car 3.32. Collocations score highest. Rare pairs are over rewarded.

KL and cross entropy D(p‖q) = Σ p log p/q ≥ 0, not symmetric, infinite if q = 0 where p > 0. H(p,q) = H(p) + D(p‖q).

Noisy channel argmax p(i) p(o|i): language model times channel model. acress → actress (0.485). BSC 10%: C = 0.531.

Conditional entropy

$$H(Y \mid X) = \textstyle\sum_x p(x)\, H(Y \mid X = x) = -\sum_{x,y} p(x, y) \log_2 p(y \mid x)$$

Chain rule $H(X, Y) = H(X) + H(Y \mid X)$

Mutual information

$$I(X : Y) = H(Y) - H(Y \mid X) = H(X) + H(Y) - H(X, Y)$$

Pointwise mutual information

$$\mathrm{PMI}(x, y) = \log_2 \frac{p(x, y)}{p(x)\, p(y)} = \log_2 \frac{\mathrm{c}(x\, y) \cdot N}{\mathrm{c}(x) \cdot \mathrm{c}(y)}$$

Kullback Leibler divergence

$$D(p \,\|\, q) = \textstyle\sum_x p(x) \log_2 \frac{p(x)}{q(x)}$$

Cross entropy

$$H(p, q) = -\textstyle\sum_x p(x) \log_2 q(x) = H(p) + D(p \,\|\, q)$$

Noisy channel decoding

$$\hat{\imath} = \operatorname{argmax}_i p(i \mid o) = \operatorname{argmax}_i p(i) \cdot p(o \mid i)$$

TRAP A student guessed $-\Sigma$ p(x,y) log p(x|y). That is not H(Y|X), but it is a correct formula for H(X|Y): the conditioning just runs the other way. In the umbrella example it gives 0.593 bits against 0.621 for H(Y|X).

TRAP PMI over rewards rare pairs. Two words seen once, together, score $\log_2$ N, more than New Delhi. Always use a minimum count, or a t-score or log likelihood ratio, alongside it.

EXAM If a model gives probability 0 to something that happens, KL and perplexity become infinite. That single fact is why every n-gram model in chapter 4 is smoothed, and why neural models use a softmax, which never outputs exactly 0.

STEP 4 NLU · Language models

4.1 N-gram models and the Markov assumption

Markov assumption An n-gram looks at n − 1 previous words. Bigram = first order Markov chain. Chain rule is exact, the n-gram is an approximation.

MLE P(w | h) = C(h w) ÷ C(h). Unigram P(w) = C(w) ÷ N. Proof: Lagrange multiplier gives the relative frequency.

The tags <s> is context only, </s> is predicted. Activity corpus: N = 17 + 5 = 22, six predicted types.

Activity answers Bigram S1 = 6/625 = 0.0096 (log −2.01773, PP 2.169), S2 = 0.072. Unigram S1 = 3.3075e-5, S2 = 1.0672e-3.

Perplexity N Use the tokens of the sentence being scored: words + </s> for PP, words for PP1. Never the training N.

Conditional estimate

$$\mathrm{m}(\text{jelly} \mid \text{peanut butter and}) = \frac{\mathrm{C}(\text{peanut butter and jelly})}{\mathrm{C}(\text{peanut butter and})}$$

Chain rule (exact)

$$\mathrm{P}(w_1 \ldots w_n) = \mathrm{P}(w_1) \cdot \mathrm{P}(w_2 \mid w_1) \cdot \mathrm{P}(w_3 \mid w_1\, w_2) \cdots \mathrm{P}(w_n \mid w_1 \ldots w_{n-1})$$

Markov assumption

$$\mathrm{P}(w_i \mid w_1 \ldots w_{i-1}) \approx \mathrm{P}(w_i \mid w_{i-n+1} \ldots w_{i-1})$$

Bigram MLE $\mathrm{P}(w_n \mid w_{n-1}) = \frac{\mathrm{C}(w_{n-1}\, w_n)}{\mathrm{C}(w_{n-1})}$

TRAP A student first answered "count jelly and divide by the total number of words". That is P(jelly), the unigram probability, which ignores the history. The question asks for the conditional probability, which needs the joint count of the whole sequence.

EXAM In class the answer to "how many context words does a 5-gram have?" came out garbled. The rule is simple: n words in total, the last is predicted, so a 5-gram has 4 context words.

TRAP Three errors in the answer sheet are worth knowing. It labels unigram factors as P(Sam | <s>), but a unigram has no condition. Its product for S1 lists three numbers and gives 3.25×10^{-5}, which comes from rounding 5/22 to 0.227. The exact value is 3.3075×10^{-5}. And many students used N = 17 or 27, or conditioned the unigram on the previous word.

EXAM PP divides the total surprise by words plus </s>, PP1 by words only, so PP1 is always the larger. The N in PP = $P^{-1/N}$ is the length of the sentence being scored, 6 for S1, not the 22 tokens of the training corpus. The professor saw that mistake in many submissions.

4.2 Perplexity and evaluating language models

Perplexity PP = $2^{\hat{H}}$ = $P^{-1/N}$. Uniform over k words gives PP = k. "He put the coffee on": 13.288 bits, $\hat{H}$ 2.658, PP 6.31.

ppl and ppl1 Same logprob. ppl divides by words − OOV − zeroprobs + sentences, ppl1 by words − OOV − zeroprobs. File ppl from totals, not an average.

Evaluation Extrinsic (task) against intrinsic (likelihood, log likelihood, perplexity). Use logs to avoid underflow below about 10^{-308}.

Rules Never train or tune on the test set. Compare perplexities only with the same test text and vocabulary. Report OOVs.

WSJ Unigram 962, bigram 170, trigram 109 (9.91, 7.41, 6.77 bits). Test perplexity is 2 to the cross entropy, which is at least the true entropy.

Perplexity

$$\mathrm{PP} = 2^{\hat{H}}, \quad \hat{H} = \frac{H_{\text{total}}}{N}, \quad \text{so} \quad \mathrm{PP} = \mathrm{m}(w_1 \ldots w_N)^{-1/N}$$

ppl

$\text{ppl} = 10^{-\text{logprob} \div (\text{words} - \text{OOVs} - \text{zeroprobs} + \text{sentences})}$

ppl1

$\text{ppl1} = 10^{-\text{logprob} \div (\text{words} - \text{OOVs} - \text{zeroprobs})}$

The three intrinsic measures

likelihood $\text{m}(\text{test})$, log likelihood $\log \text{m}(\text{test})$, $\text{PP} = \text{m}(\text{test})^{-1/N}$

Test set cross entropy

$\hat{H} = -\frac{1}{N} \log_2 \text{m}(w_1 \ldots w_N) \approx H(p, \text{m}) = H(p) + D(p \,\|\, \text{m})$

EXAM For long documents PP and PP1 converge, since there is one end tag per twenty or more words. For three word toy sentences they differ a lot (1.93 against 2.40). In an exam, say which N you used and never compare PP from one model with PP1 from another.

4.3 Unseen words and smoothing

Two zeros OOV word: map to <UNK>. Unseen n-gram of known words: smooth, back off or interpolate. One zero factor gives P = 0 and PP = ∞.

Laplace and add-k (C + k) ÷ (C(h) + kV). V = 7 for the activity. S3 Laplace bigram = 1/17010, PP 5.07. Sums to 1 by adding kV.

Discounting Absolute: max(C − d, 0)/C(h) + λP(w), λ = d N_{1+}/C(h). Good-Turing: c* = (c+1)N_{c+1}/N_c, unseen N_1/N. KN uses continuation counts.

Backoff Use the highest order with evidence, discounted, with α so the row sums to 1. Problem: "John eats blue", unseen is not always sparse.

Interpolation Σλ P = 1 when Σλ = 1. Tune λ on dev (S3: best near 0.48, PP 6.43), report test once. EM for many λ.

Laplace (add-one)

$P_{\text{Lap}}(w) = \frac{C(w) + 1}{N + V}, \quad P_{\text{Lap}}(w \mid h) = \frac{C(h\,w) + 1}{C(h) + V}$

Absolute discounting

$P(w \mid h) = \frac{\max(C(h\,w) - d, 0)}{C(h)} + \lambda(h)\, P(w), \quad \lambda(h) = \frac{d \cdot N_{1+}(h\,\bullet)}{C(h)}$

Kneser-Ney

$P_{\text{cont}}(w) = \frac{|\{v : C(v\,w) > 0\}|}{\text{number of bigram types}}$

Linear interpolation

$\hat{P}(w \mid h) = \lambda_3 P(w \mid u\,v) + \lambda_2 P(w \mid v) + \lambda_1 P(w), \quad \Sigma\,\lambda = 1, \lambda \geq 0$

TRAP The answer sheet got this one wrong twice. It wrote P(linguistics | like) = 0.01 instead of 1/10. Then it multiplied in the unigram values 0.034 and 0.207 in brackets, which is neither a Laplace bigram nor a proper backoff, and does not sum to 1. SRILM's vocabulary also contains <s> (its file lists 8 unigrams), so its add-one numbers can differ slightly. In the exam, state your V.

EXAM Chen and Goodman (1998) compared all of these. Modified Kneser-Ney with interpolation was the most accurate classic n-gram smoother, and it is still the default in toolkits such as KenLM. In assignment 2 it also won on the Brown corpus.

4.4 The SRILM toolkit and assignment 2

Workflow ngram-count -write (counts), ngram-count -lm (model), ngram -lm -ppl -debug 2 (scores). Add -unk to map unknown words.

ARPA $\log_{10}$ probabilities and backoff weights. −99 = log 0. Toy header ngram 1=8, ngram 2=14. $\log_{10}(5/22) = -0.6434527$.

Default smoothing Good-Turing with Katz backoff. Flags: -addsmooth k, -cdiscount, -wbdiscount, -kndiscount, -interpolate.

Brown results Bigram ppl: add-one 2493, GT 642, WB 397, absolute 392, KN 352. Trigram lowers all except add-one.

Stop words and tuning Removing punctuation and stop words raises ppl (397, 611, 3235), a trend only. Best k 0.001, λ 0.7, dev 581, test 681.

STEP 5 NLU · Tagging and syntax

5.1 Part of speech tagging

The task Give each word its part of speech. Two cues: the word itself and its context. book: noun or verb. can: modal, noun or verb.

Tagsets Traditional about 8, universal 12, UPOS 17, Penn Treebank 45, Brown about 87. More tags, harder but more informative.

Ambiguity numbers About 11% of Brown types but 40% of tokens are ambiguous. Most frequent tag baseline about 92%, good taggers 97% and above.

Kinds of tagger Look up with backoff, generative HMM (P(W, T)), discriminative MaxEnt, perceptron, CRF (P(T | W)), neural BiLSTM and BERT.

Generative and discriminative argmax P(T | W) = argmax P(W, T) because P(W) is fixed. Not the same axis as supervised and unsupervised.

The problem statement

given words $W = w_1 \ldots w_n$, find tags $\hat{T} = \text{argmax}_T\, P(T \mid W)$

Generative

$\hat{T} = \text{argmax}_T\, P(W, T) = \text{argmax}_T\, P(T) \cdot P(W \mid T)$

Discriminative $\hat{T} = \text{argmax}_T\, P(T \mid W)$

5.2 Finite state machines and the HMM

FSA Five tuple (Q, Σ, q0, δ, F). Accept only if every symbol has an arrow and the machine ends in a final state. Sheep talk is /baa+!/.

HMM Five tuple (Q, A, O, B, π). Tags are hidden, words are observed. A probabilistic Moore machine: states emit words.

Joint probability P(W, T) = π(t1) b(w1 | t1) × product over i ≥ 2 of a(t(i−1) → ti) b(wi | ti). 2n factors. Markov and output independence assumptions.

Validity Each row of A, each row of B and π sums to 1. A missing cell is 1 minus the rest of its row.

Training Count starts, tag bigrams and (tag, word) pairs, then divide by row totals. Zeros need smoothing.

The FSA five tuple $(Q, \Sigma, q_0, \delta, F)$

The HMM five tuple (Q, A, O, B, π)

Joint probability of words and tags

$P(W, T) = \pi(t_1)\, b_{t_1}(w_1) \prod_{i=2}^{n} a_{t_{i-1} t_i}\, b_{t_i}(w_i)$

Transition

$a(t \to u) = \frac{C(t\,u)}{C(t \text{ followed by any tag})}$

Emission $b(w \mid t) = \frac{C(w \text{ tagged } t)}{C(t)}$

TRAP Do not confuse an HMM with a bigram word model. In the bigram model the thing you condition on, the previous word, is visible. In the HMM it is a hidden tag, so to find tags you must search (Viterbi) or sum (forward) over them.

EXAM Exam questions often hide one cell. Fill it with 1 minus the rest of its row (problem G2). The rows are what sum to 1, never the columns: P(N | A) + P(N | N) = 2/3 + 9/10 is not 1 and does not need to be.

TRAP Two different killer clown HMMs appear in the deck. Slides 23 and 24 give P(killer clown, N N) = 1/2 × 1/3 × 9/10 × 1/3 = 0.05. Slide 27 gives 0.045. Same winner, different number. Always use the tables the question gives you (problem G4).

5.3 Decoding with Viterbi

Search size Brute force scores q^T sequences. Viterbi does about T × q² steps. PTB 45 tags, 25 words: 2.1 × 10^41 against 50,625.

Viterbi recursion v1(j) = πj bj(w1). vt(j) = max over i of vt−1(i) aij bj(wt). Keep back pointers, take the best last cell and trace back.

Forward Same table with sum: P(W) = sum of the last column. Posterior P(T | W) = P(W, T) ÷ P(W). they can fish: 0.073164 and 0.7233.

Why it works Principle of optimality: under the Markov assumption the best path passes through the best partial path into each cell.

Ties and trigrams Ties: pick either, or return k best. Trigram HMM (TnT): O(T q³), needs smoothing, about 96.7% on PTB.

The decoding objective

$\hat{T} = \text{argmax}_{t_1 \ldots t_n} \prod_i P(w_i \mid t_i) \cdot P(t_i \mid t_{i-1})$

Viterbi recursion

$v_1(j) = \pi_j\, b_j(w_1) \qquad v_t(j) = \max_i v_{t-1}(i) \cdot a_{ij} \cdot b_j(w_t)$

Forward recursion

$\alpha_1(j) = \pi_j\, b_j(w_1) \qquad \alpha_t(j) = \Sigma_i\, \alpha_{t-1}(i) \cdot a_{ij} \cdot b_j(w_t)$

EXAM Exam recipe. 1: write π, A and B as tables and check the rows. 2: if T ≤ 3 and q ≤ 2, enumerating all paths is fine, show every product. 3: otherwise fill a q × T table column by column, writing the winning predecessor in each cell. 4: back trace from the best last cell. 5: state the tag sequence and its probability. Strike out zero emissions first.

5.4 Chunking and named entities

Chunk A flat, non overlapping group of words such as [the yellow dog]. Chunking is shallow parsing. Constituents nest, chunks do not.

IOB B-X begins, I-X continues, O is outside. 2k + 1 labels. Turns span finding into tagging with the same models.

Decoding IOB A span is a B-X plus the following I-X tags. An I-X after O or another type is repaired as B-X.

NER PER, ORG, LOC, DATE, MONEY. Perceptron, CRF, BiLSTM CRF and BERT. Features: capitals, shape, gazetteers, neighbours.

Scoring Entity level precision, recall and F1 with exact start, end and type. Token accuracy is inflated by O.

IOB tags

B-*X* begins a chunk of type X **I-*X*** is inside it **O** is outside every chunk

Entity level F1

$$P = \frac{\text{correct spans}}{\text{predicted spans}} \quad R = \frac{\text{correct spans}}{\text{gold spans}}$$

$$F1 = \frac{2PR}{P + R}$$

5.5 Constituency and dependency syntax

Constituents Groups that act as one unit. Tests: movement, substitution by a pro form, fragment answer, coordination, clefting.

CFG Four tuple (N, Σ, R, S). Rules A → β. Bracket notation stores trees. Penn Treebank supplies trees and rule counts.

Dependencies Head → dependent arcs with labels (nsubj, obj, det). A tree: one head per word, n arcs with ROOT, no cycles. UD makes content words heads.

PCFG P(tree) = product of rule probabilities, rules for each non terminal sum to 1. Elephant: VP attach 0.01575 against 0.0105, ratio 1.5.

Ambiguity growth k PPs give Catalan C(k + 1) parses: 2, 5, 14, 42. CKY parses in O(n³).

Bracket notation

(S (NP (N Fed)) (VP (V raises) (NP (N interest) (N rates))))

CFG four tuple (N, Σ, R, S)

Probabilistic CFG

$$P(\text{tree}) = \prod_{\text{rules } A \to \beta \text{ used}} P(\beta \mid A), \quad \text{with } \Sigma_\beta P(\beta \mid A) = 1 \text{ for each } A$$

EXAM A student asked whether this is a DAG, as in compilers. It is stricter: a tree. Every word has exactly one head, there are no cycles, and a sentence of n words has exactly n arcs counting the ROOT arc.

STEP 6 NLU · Exam revision

6.1 Exam revision: from school to research universities

Type token ratio $TTR = \frac{\text{number of types}}{\text{number of tokens}}$

Zipf's law $f \times r \approx C$

TRAP Do not mix tokens and types. "How many words" is ambiguous in an exam. Say which you counted.

TRAP State your convention for <s> and </s> before computing. Counting </s> as a predicted token changes both the probability and the N in perplexity.

EXAM Trap options to expect: counting <s> as a predicted token, mixing log base 2 and 10, forgetting to add V in the Laplace denominator, treating columns of an HMM table as summing to 1, and calling a chunk a constituent.

EXAM In derivations, name each assumption when you use it: Markov, output independence, a fixed vocabulary, independent sentences. Examiners often ask what breaks when one fails.

STEP 1 NLP · Neurons and learning

1.1 From regression to learning by gradient descent

Model and cost A model has knobs (weights w and bias b). The squared error cost C = ∑(ŷ − y)² scores a guess. For the lecture data w = 1, b = 1.37 gives 0.67.

Gradient descent Move each knob against its slope: θ ← θ − η ∂C/∂θ. Too large η overshoots or diverges, too small η is slow.

One knob example NN() = b, cost (b − 4)², slope 2(b − 4). The distance to 4 is multiplied by 1 − 2η each step, η = 0.5 lands in one step.

Partial derivatives ∂C/∂w = ∑2(wx + b − y)x and ∂C/∂b = ∑2(wx + b − y). The cost over (w, b) is a bowl with one minimum.

Chain rule For ŷ = tanh(wx + b): ∂L/∂w = 2(ŷ − y)(1 − ŷ²)x. A negative gradient raises w, but a large step can still raise the cost.

Squared error cost

$$C(w, b) = \sum_i (wx_i + b - y_i)^2$$

Gradient descent update $b \leftarrow b - \eta \cdot \frac{dC}{db}$

Partial derivatives of the squared error

$$\frac{\partial C}{\partial w} = \sum 2(wx + b - y) \cdot x \quad \frac{\partial C}{\partial b} = \sum 2(wx + b - y)$$

Chain rule for one neuron

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w} = 2(\hat{y} - y) \cdot (1 - \hat{y}^2) \cdot x$$

TRAP A negative slope does not by itself promise a lower cost after the step. If η is too large the step overshoots and the cost can go up. The lecture asked exactly this.

EXAM Expect "if ∂C/∂w is negative, does w increase or decrease after the update?" It increases, because w − η × (negative) adds a positive amount.

EXAM Formulas for derivatives are given in the exam if you need them. If none is given, the question can be answered without one. This was said in class.

1.2 One neuron, the sigmoid and backpropagation

Neuron NN(x) = σ(w·x + b) with σ(z) = 1 ÷ (1 + e^{-z}). NN(2, 1) with w = (0.5, 0.2), b = 0.3 is 0.82. Output above 0.5 means class 1.

Boundary σ(z) = 0.5 exactly at z = 0, so one neuron draws a straight line. It cannot separate XOR.

Chain rule ∂cost/∂w_1 = 2(pred − target) · σ(z) (1 − σ(z)) · x_1. For b the last factor is 1. Update w ← w − η · gradient.

Notebook 10,000 random single flower steps with η = 0.1 from random weights. The mystery flower (4.5, 1) comes out 0.99, red.

Loss choice Squared error with a sigmoid learns slowly when confidently wrong. Cross entropy gives the clean gradient (σ − y)x.

One neuron with two inputs

$$NN(m_1, m_2) = \sigma(w_1 m_1 + w_2 m_2 + b)$$

Sigmoid $\sigma(z) = \frac{1}{1 + e^{-z}}$

The three links

$$\frac{\partial \text{cost}}{\partial w_1} = \frac{\partial \text{cost}}{\partial \text{pred}} \cdot \frac{\partial \text{pred}}{\partial z} \cdot \frac{\partial z}{\partial w_1} = 2(\text{pred} - \text{target}) \cdot \sigma'(z) \cdot \text{length}$$

Slope of the sigmoid $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

TRAP Do not forget the σ'(z) link. Without it you are differentiating a straight line, not a neuron.

EXAM A typical question gives ŷ = σ(wx + b) or tanh(wx + b), one data point and the cost, and asks for ∂L/∂w, the predicted value, and the direction w moves.

1.3 Activation functions: why networks need a bend

Why Without activations $W_2W_1x = Wx$: depth adds nothing. Non-linearities bend space and make networks universal function approximators.

Classic units Threshold (no gradient), sigmoid (0, 1), tanh = 2σ(2z) − 1 in (−1, 1), hard tanh. Sigmoid and tanh saturate.

ReLU max(z, 0). Slope 1 for z > 0 and 0 for z < 0 (dead neurons). Leaky and parametric ReLU keep a small negative slope.

Smooth gates swish = z·σ(z), GELU = z·Φ(z) ≈ z·σ(1.702z), used in BERT and RoBERTa.

Trade off More non-linear layers mean more expressive power but harder optimisation and overfitting risk.

Linear layers collapse

$$W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x} = W\mathbf{x}$$

tanh from the sigmoid $\tanh(z) = 2\sigma(2z) - 1$

ReLU and leaky ReLU

$$\text{ReLU}(z) = \max(z, 0) \quad \text{leaky}(z) = z \text{ for } z > 0,\ a \cdot z \text{ otherwise}$$

GELU

$$\text{GELU}(z) = z \cdot P(X \le z),\ X \sim N(0, 1) \quad \approx z \cdot \sigma(1.702\, z)$$

TRAP More non-linear layers give more expressive power but also harder optimisation and a higher risk of overfitting. The slide closes with exactly this warning.

EXAM Expect "what is the gradient of ReLU for x < 0?" Answer 0. Expect also "why tanh over sigmoid in hidden layers?" Answer: zero centred and stronger slope.

STEP 2 NLP · Word vectors

2.1 From symbols to word vectors

One-hot Length = vocabulary size (500,000+), one 1. Every pair is orthogonal (dot 0, distance √2), so one-hot vectors carry no similarity.

Distributional semantics A word's meaning comes from the words near it (Firth 1957). Context = words within ±m positions.

Word vectors Dense, short, learned so that words in similar contexts get similar vectors. Also called embeddings. A distributed representation.

Similarity Dot product and cosine = a·b ÷ (‖a‖‖b‖). banking against monetary: cosine 0.857.

WordNet fails Incomplete synonym lists, similarity is graded not binary, and context changes meaning.

Every pair is orthogonal
$\text{hotel} \cdot \text{motel} = 0 \quad \text{and} \quad \|\text{hotel} - \text{motel}\| = \sqrt{2}$

Dot product $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$

Cosine similarity $\cos(\mathbf{a}, \mathbf{b}) = \dfrac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$

TRAP Cosine similarity is not a probability and can be negative. Do not compare a cosine with a probability.

2.2 Word2vec: skip-gram with the full softmax

Word2vec idea Slide a window over the text. Use the centre word to predict each outside word, and adjust the vectors to make true neighbours probable (Mikolov et al. 2013).

Objective $J(\theta) = -(1/T) \sum_t \sum_j \log P(w_{t+j} \mid w_t)$: log, minus sign, average.

Prediction $P(o \mid c) = \exp(u_o \cdot v_c) \div \sum_w \exp(u_w \cdot v_c)$: dot product, exp, normalise. Two vectors per word, two roles.

Gradient $\partial J / \partial v_c = -u_o + \sum P(w \mid c) u_w$, observed minus expected. Trained with SGD, one window at a time.

Variants and size Skip-gram predicts context from centre, CBOW centre from averaged context. Full softmax has 2Vd parameters: 6,000,000 for V = 10,000, d = 300.

Objective to minimise
$J(\theta) = -\dfrac{1}{T} \sum_t \sum_j \log P(w_{t+j} \mid w_t, \theta)$

Skip-gram prediction function
$P(o \mid c) = \dfrac{\exp(u_o \cdot v_c)}{\sum_{w \in V} \exp(u_w \cdot v_c)}$

Gradient for the centre vector
$\dfrac{\partial J}{\partial v_c} = -u_o + \sum_{w \in V} P(w \mid c) \, u_w$

Parameters of skip-gram with full softmax
$|E| + |W| = V \cdot d + d \cdot V = 2Vd$

EXAM Sigmoid or softmax? The lecture answer: sigmoid for a yes or no (binary) decision, softmax for picking one of many classes. Full word2vec picks one word out of V, so it uses softmax. Negative sampling turns it into many yes or no decisions, so it uses sigmoids.

TRAP With full softmax the error vector p − y has V entries, so every single update touches all V output vectors. That is the cost negative sampling removes.

2.3 Skip-gram with negative sampling

Why The softmax denominator sums over the whole vocabulary. Negative sampling replaces it with 1 + K binary logistic regressions.

Objective $J_{neg} = -\log \sigma(u_o \cdot v_c) - \sum_k \log \sigma(-u_k \cdot v_c)$, $\sigma(-x) = 1 - \sigma(x)$.

Noise distribution $P_n(w) \propto U(w)^{3/4}$: frequent words like "the" drop, rare words rise. The true context is never used as noise.

Counting question Sigmoids per centre word = (1 + K) × number of context words: window 2, K = 10 gives 44.

Notebook K = 3, counts$^{0.75}$, 500 epochs: cats and dogs become nearest neighbours (cosine 0.98). Optimum: u·v = PMI − log K.

Loss for one pair
$J_{neg}(o, v_c, U) = -\log \sigma(u_o \cdot v_c) - \sum_{k=1}^{K} \log \sigma(-u_k \cdot v_c)$

Gradients
$\dfrac{\partial J}{\partial v_c} = (\sigma(u_o \cdot v_c) - 1) u_o + \sum_k \sigma(u_k \cdot v_c) u_k,$
$\dfrac{\partial J}{\partial u_k} = \sigma(u_k \cdot v_c) v_c$

Noise distribution $P_n(w) = \dfrac{U(w)^{3/4}}{\sum_{w'} U(w')^{3/4}}$

EXAM Count positives and negatives per context word, then multiply by the number of context words. Read whether the question gives K per pair or per centre word.

2.4 Co-occurrence counts, SVD and GloVe

Co-occurrence matrix X_{ij} counts word j inside the window of word i. Window matrices give word space, word by document matrices give topics (LSA).

SVD $X = U \Sigma V^T$, keep the top k singular values for k-dimensional vectors. Raw counts need log, caps or removing function words first.

GloVe idea Meaning is in ratios P(k | i) ÷ P(k | j): much more than 1, much less than 1, or about 1 (ice, steam, solid, gas, water, fashion).

GloVe loss $J = \sum f(X_{ij})(w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$, $f(x) = (x \div x_{max})^{\alpha}$ below x_{max} else 1, x_{max} = 100, α = 3/4.

Notebook Window 2 with 1 ÷ distance, d = 20, x_{max} = 10, final vectors w + w̃. 882 parameters for 80 pairs, so the loss hits 0 and the tiny model memorises.

Window co-occurrence
X_{ij} = number of times word j appears within m words of word i

SVD $X = U \, \Sigma \, V^T$

Truncated SVD $X_k = U_k \, \Sigma_k \, V_k^T$

GloVe model equation
$w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}$

GloVe objective
$J = \sum_{i,j} f(X_{ij}) \, (w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$

Weighting function
$f(x) = (x \div x_{max})^{\alpha}$ if $x < x_{max}$, else 1

TRAP SVD costs about $O(V^3)$ time for a V × V matrix and must be redone when new words arrive. That is one reason prediction based methods such as word2vec took over, and GloVe tries to combine the two.

EXAM Exams like the table: given two probabilities, compute the ratio and say whether the probe word is an ice word, a steam word or neutral. 8.9 is an ice word, 0.085 a steam word, 1.36 and 0.96 neutral.

TRAP The loss is a weighted squared error on log counts, not a cross entropy. Students often write −log P like word2vec.

2.5 Evaluating and improving embeddings

Intrinsic and extrinsic Intrinsic: fast subtasks (relatedness with Spearman, analogy, categorisation with purity, selectional preference). Extrinsic: score of a real task.

Using pre-trained vectors Fixed (fewer parameters, no adaptation), fine-tuned (adapts, may forget), concatenated (both, bigger). No embedding wins on every test.

Subwords fastText: v_w = sum of character n-gram vectors with marks < >, n = 3 to 6. "where" has 14 n-grams. Handles rare and unseen words.

Cross-lingual Map spaces with W, often orthogonal (Procrustes $W = UV^T$). Anchors: a dictionary, identical strings (Artetxe) or none (adversarial, Zhang).

Sparsity and bias Sparse: add $\lambda \|w\|_1$. Debias: bias direction g from pairs, neutralise v' = v − (v·g)g, then equalise. Bias is reduced, not removed.

Spearman correlation (no ties)
$\rho = 1 - \dfrac{6 \sum d_i^2}{n(n^2 - 1)}$

Analogy
$d = \text{argmax}_{w \notin \{a, b, c\}} \cos(w, b - a + c)$

fastText word vector $v_w = z_w + \sum_{g \in G(w)} z_g$

Linear mapping $\min_W \sum_i \|W x_i - y_i\|^2$

Sparsity penalty $J_{sparse} = J + \lambda \sum_i \|w_i\|_1$

Neutralise $v' = v - (v \cdot g) \, g$

EXAM A Spearman question gives two lists of scores. Rank both lists first, then use the formula. Pearson uses the raw scores instead.

TRAP The slide lists the trigrams of "playing" as <pl, pla, lay, ayi, yin, ing, ng>: 7 trigrams. The last one is ng>, the suffix shared by every -ing word.

TRAP Perfect debiasing remains hard. The slide says embeddings become more balanced, not unbiased. A balanced or filtered corpus is the other approach.

STEP 3

NLP · Language models and RNNs

3.1 Language models and recurrent networks

Language model Predicts P(next word | previous words). Chain rule gives the probability of a text. n-gram models keep only n − 1 words and count.

n-gram problems Sparsity (fix: smoothing, backoff), storage, and a context that cannot exceed about 5 words. 4-gram example: 400 ÷ 1000 = 0.4.

RNN $h^{(t)} = \tanh(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$, $\hat{y} = \text{softmax}(Uh + b_2)$. Same weights at each step. One-hot 5, hidden 6: 96 parameters.

Training $J^{(t)} = -\ln \hat{y}$ of the true next word, averaged over steps. Teacher forcing feeds gold words. −ln 0.6 = 0.51, −ln 0.05 = 2.99.

Perplexity PP = exp(J), lower is better. PP = k means choosing among about k words: 1.44 confident, 4.07 confused.

Next word distribution $P(x^{(t+1)} \mid x^{(t)}, ..., x^{(1)})$

n-gram (Markov) assumption

$$P(x^{(t+1)} \mid x^{(t)}, ..., x^{(1)}) \approx P(x^{(t+1)} \mid x^{(t)}, ..., x^{(t-n+2)}) \approx \frac{\text{count}(x^{(t-n+2)} \dots x^{(t+1)})}{\text{count}(x^{(t-n+2)} \dots x^{(t)})}$$

Fixed window model

$$e = [e^{(1)}, e^{(2)}, e^{(3)}, e^{(4)}], \quad h = f(We + b_1), \quad \hat{y} = \text{softmax}(Uh + b_2)$$

RNN hidden state

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

RNN output

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

Loss at step t

$$J^{(t)}(\theta) = -\sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)} = -\log \hat{y}^{(t)}_{x^{(t+1)}}$$

Overall loss $J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta)$

Perplexity

$$PP = \prod_{t=1}^{T} \left(\frac{1}{P_{LM}(x^{(t+1)} \mid x^{(t)}, ..., x^{(1)})}\right)^{1/T} = \exp(J(\theta))$$

EXAM Questions often ask for an n-gram probability from counts and then what happens when a count is 0. Say "sparsity", name smoothing and backoff, and give the Markov assumption.

TRAP The RNN parameter count never depends on the sentence length T. A common wrong answer multiplies by T.

3.2 Sequence labelling, NER and bidirectional RNNs

Sequence labelling One softmax over K tags per word, trained with cross-entropy. POS tagging is the classic case. RNN uses: generate, label, classify.

NER Find spans that are proper names and their type: PER, LOC, ORG, GPE (also dates, prices). Hard because of segmentation and type ambiguity.

BIO B starts a span, I continues it, O is outside. 2n + 1 tags (IO n + 1, BIOES 4n + 1). Score NER with exact span F1.

Parameter answers E·H + H·H + H·K: 300, 50, 50 tags gives 20,000. NER with 5 types: 11 labels, 18,050. BiRNN doubles the RNN part and reads 2H.

BiRNN $h^{(t)} = [h\rightarrow^{(t)}, h\leftarrow^{(t)}]$: each word sees the whole sentence. Not for language modelling. Sentence vector: final, mean or max state.

Tag distribution at step t

$$\hat{y}^{(t)} = \text{softmax}(V h^{(t)}) \in \mathbb{R}^K$$

Number of BIO tags $2n + 1$

Span precision and recall

$$P = \frac{\text{correct spans}}{\text{predicted spans}}, \quad R = \frac{\text{correct spans}}{\text{gold spans}}$$

RNN tagger parameters (no biases)

$$E{\cdot}H + H{\cdot}H + H{\cdot}K$$

Joined state $h^{(t)} = [h_{\rightarrow}{}^{(t)}, h_{\leftarrow}{}^{(t)}] \in \mathbb{R}^{2H}$

TRAP Why B is needed: with IO tags "[ORG Google] [ORG Microsoft]" becomes I-ORG I-ORG and looks like one entity. B marks the start of the second one.

EXAM State what you count. If the question says "excluding bias terms" and does not mention the embedding table, count only the RNN and output weights, as the lecture answer does.

3.3 Vanishing gradients, LSTM and GRU

Vanishing and exploding BPTT multiplies one Jacobian $\text{diag}(\sigma')W_h$ per step: below 1 it vanishes, above 1 it explodes. Fixes: ReLU, initialisation, residual links, LSTM/GRU, clipping for exploding.

LSTM equations f, i, o = σ(Wh + Ux + b), $\tilde{c}$ = tanh(...), $c = f \odot c_{old} + i \odot \tilde{c}$, $h = o \odot \tanh(c)$. Parameters 4H(E + H) + 4H.

Why LSTM works Additive cell path: $\partial c^{(t)} / \partial c^{(t-1)} \approx f$. With f near 1 information and gradients survive many steps.

GRU z and r gates, $\tilde{h} = \tanh(Wx + U(r \odot h) + b)$, $h = (1 - z) \odot h + z \odot \tilde{h}$. Parameters 3H(E + H). Faster, fewer parameters.

BiLSTM answer E = 60, H = 32 each way, 4 tags: 2 × 4 × 32 × 92 + 64 × 4 = 23,808. RNNs cannot parallelise over time, so attention and transformers followed.

One step back

$$\frac{\partial h^{(t)}}{\partial h^{(t-1)}} = \text{diag}(\sigma'(W_h h^{(t-1)} + W_x x^{(t)} + b)) W_h$$

Many steps back

$$\frac{\partial J^{(t)}}{\partial h^{(j)}} = \frac{\partial J^{(t)}}{\partial h^{(t)}} \prod_{j < k \le t} \frac{\partial h^{(k)}}{\partial h^{(k-1)}}$$

Forget gate $f^{(t)} = \sigma(W_f h^{(t-1)} + U_f x^{(t)} + b_f)$

Cell state $c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$

Hidden state $h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$

LSTM layer parameters $4 \cdot H \cdot (E + H) + 4H$

Direct cell path $\frac{\partial c^{(t)}}{\partial c^{(t-1)}} \approx \text{diag}(f^{(t)})$

Update gate $z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$

New state $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

TRAP Vanishing gradients mean the model does not learn long range dependencies. They do not mean the loss is 0 or that training stops at once. Exploding gradients show up as NaN losses.

EXAM An LSTM does not guarantee there is no vanishing or exploding gradient. It makes it much easier for the model to learn long range dependencies. In practice it handles about 100 steps well where a plain RNN manages about 7.

STEP 4

NLP · Translation, attention and decoding

4.1 Sequence-to-sequence models and machine translation

Seq2seq Encoder RNN reads x, its last state starts the decoder RNN, which writes y one word at a time and feeds its outputs back in at test time.

Conditional LM $P(y \mid x) = \prod P(y_t \mid y_{<t}, x)$. Language model: predicts the next target word. Conditional: on the whole source. Trained on a parallel corpus.

Uses Translation, summarisation, dialogue, parsing, code generation: any task whose input and output are sequences.

Bottleneck The whole source passes through one vector, so long sentences lose information. Attention fixes it.

Low-resource and parameters Transfer, multilingual, back translation, augmentation, pivot, unsupervised. Spanish to French count with stated assumptions: 38,005,760, nearly all in embeddings and output.

Decoder $s_t = \text{RNN}_{dec}(s_{t-1}, y_{t-1}), \quad s_0 = h_n^{enc}$

NMT probability

$$P(y \mid x) = \prod_{t=1}^{T} P(y_t \mid y_1, ..., y_{t-1}, x)$$

Training loss

$$J = \frac{1}{T} \sum_{t=1}^{T} -\log P(y_t^{gold} \mid y_{<t}^{gold}, x)$$

EXAM In the exam, name both parts: "language model" because it predicts the next target word, "conditional" because it conditions on the whole source x.

EXAM Most of the parameters sit in the embeddings and the output layer, about 37.8 million of 38 million. The RNNs themselves are tiny. Variants change the total: concatenating the context to the decoder input adds 128 × 256 = 32,768, and a decoder with the same size as the encoder needs no bridge or projection.

4.2 Attention in encoder-decoder models

Attention idea At each decoder step, look directly at all encoder states and focus on the relevant ones. A dynamic context per step solves the bottleneck.

Equations $e_i = s_t^T h_i$, $\alpha = \text{softmax}(e)$, $a_t = \sum \alpha_i h_i$, then predict from $[a_t, s_t]$.

Lecture problem Scores 2, −5, 3, −2, weights 0.2676, 0.0002, 0.7273, 0.0049, context ≈ [1.262, −0.722].

Variants Dot (same size), multiplicative $s^T W h$, reduced rank $(Us)^T(Vh)$, additive $v^T \tanh(W_1 h + W_2 s)$, scaled by $1 \div \sqrt{d}$.

Benefits Better NMT, no bottleneck, gradient shortcuts, interpretable weights and soft alignment learned for free. Leads to self-attention and transformers.

1. Scores $e^t = [s_t^T h_1, ..., s_t^T h_N] \in \mathbb{R}^N$

2. Attention distribution

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

3. Attention output (context)

$$a_t = \sum_{i=1}^{N} \alpha_i^t h_i \in \mathbb{R}^d$$

Basic dot product $e_i = s^T h_i$

Multiplicative (bilinear, Luong)

$e_i = s^T W h_i, \quad W \in \mathbb{R}^{d_s \times d_h}$

Additive (Bahdanau)

$e_i = v^T \tanh(W_1 h_i + W_2 s)$

TRAP The weights depend on the decoder state, so they change at every output step. That is what makes the context dynamic. A common exam slip is to compute one set of weights for the whole sentence.

EXAM Show the scores, the softmax and the weighted sum as three separate lines. Keep 4 decimals in the weights, and check that they add to 1 (0.2676 + 0.0002 + 0.7273 + 0.0049 = 1.0000).

4.3 Decoding: greedy, beam search and sampling

Greedy Pick argmax at each step. Fast, cannot undo decisions. Slide tree: greedy "a cat" 0.20, optimal "the cat" 0.36.

Beam search Keep the k best hypotheses by ∑ log P, expand each, keep the top k again. k = 1 is greedy. Not guaranteed optimal, far cheaper than $|V|^T$.

Stopping and length A hypothesis that emits <END> is set aside. Stop at time T or after n complete hypotheses. Pick the best by (1 ÷ t) ∑ log P.

Sampling Temperature softmax(s ÷ T): low T sharp, high T flat. Top-k keeps k words, top-p the smallest set with mass ≥ p. Adds diversity.

Trade off Greedy and beam are deterministic and can repeat in open ended text. Sampling trades some quality for diversity.

Greedy step $\hat{y}_t = \text{argmax}_{w \in V} P(w \mid \hat{y}_{<t}, x)$

What we really want

$\hat{y} = \text{argmax}_y P(y \mid x) = \text{argmax}_y \prod_t P(y_t \mid y_{<t}, x)$

Hypothesis score

$\text{score}(y_1, \ldots, y_t) = \log P(y_{1:t} \mid x) = \sum_{i=1}^{t} \log P(y_i \mid y_{<i}, x)$

One beam step

expand each of the k hypotheses by every word, keep the top k of the $k \cdot |V|$ candidates

Length normalised score

$\frac{1}{t} \sum_{i=1}^{t} \log P(y_i \mid y_{<i}, x)$

Temperature $P_T(w) = \frac{\exp(s_w \div T)}{\sum_{w'} \exp(s_{w'} \div T)}$

Top-k

sample only from the k most probable words, renormalised

Top-p (nucleus)

sample from the smallest set V_p with $\sum_{w \in V_p} P(w) \geq p$

EXAM Greedy is fast and often fine, but it is not optimal. Exams ask for an example where greedy fails: show a first choice with a high probability followed by low probabilities, against a lower first choice followed by a high one.

TRAP Beam search is not guaranteed to find the optimal solution. It is much more efficient than exhaustive search, and a larger k gets closer to the optimum at a higher cost.